

IsotopeIQ Satellite — Administrator Guide

Table of Contents

1. [Overview](#)
 2. [Prerequisites](#)
 3. [Installation](#)
 4. [Pre-Built Bundle \(Offline\)](#)
 5. [From Source](#)
 6. [Accessing the Application](#)
 7. [Environment Configuration](#)
 8. [Starting and Stopping Services](#)
 9. [Authentication Configuration](#)
 10. [Local Authentication](#)
 11. [LDAP](#)
 12. [SAML 2.0](#)
 13. [Notification Configuration](#)
 14. [Service Management Reference](#)
 15. [Data Retention](#)
 16. [Upgrading](#)
 17. [Backup and Restore](#)
 18. [Troubleshooting](#)
 19. [Security Hardening](#)
 20. [Architecture Reference](#)
 21. [Agent Installation \(Offline\)](#)
 - [Files to Transfer](#)
 - [Generating the Config File](#)
 - [Linux](#)
 - [macOS](#)
 - [Windows](#)
 - [Default Agent Port](#)
 - [Uninstalling](#)
 - [Using the Python Source](#)
-

1. Overview

IsotopeIQ Satellite is a configuration collection, baseline, and drift-detection platform. It connects to managed devices (Linux, Windows, macOS, network devices, SNMP systems), collects their configuration state, normalizes the output into a canonical JSON structure, stores baselines, and continuously alerts when device configuration drifts from those baselines.

Core workflow:

Service stack:

Container	Role	Default Port
db	PostgreSQL 16	5432 (internal)
redis	Celery broker / cache	6379 (internal)
backend	Django REST API	8000 (internal)
celery_worker	Job execution	—
celery_beat	Job scheduler	—
frontend	Vue 3 / Vite UI	80 (internal)
nginx	Reverse proxy / TLS	80, 443

2. Prerequisites

- **Docker** 24+ with the Compose plugin (`docker compose` — not legacy `docker-compose`)
- **Git** — only required for source-based installation (§3.2)
- **Python 3** on the host — only required to generate `FIELD_ENCRYPTION_KEY` (see §4.1)
- Network access from the Satellite host to all managed devices on their management ports (SSH 22, WinRM 5985/5986, SNMP 161, etc.)

Verify Docker is ready:

```
docker --version
docker compose version
```

3. Installation

Two installation paths are available:

Path	When to use
§3.1 Pre-built bundle	Air-gapped / offline servers, no build tools required
§3.2 From source	Development or when building images on the server

3.1 Pre-Built Bundle Installation (Offline)

This path uses a self-contained archive that includes all Docker images. No internet access, Git, or build tools are required on the target server — only Docker.

3.1.1 Transfer the Bundle

Copy the bundle archive to the server by whatever means is available (USB, SCP, shared drive, etc.):

```
scp isotopeiq-satellite-docker-<date>.tar.gz user@server:~
```

3.1.2 Extract and Configure

```
tar -xzf isotopeiq-satellite-docker-<date>.tar.gz
cd isotopeiq-satellite-docker-<date>
cp .env.example .env
```

Edit `.env` and fill in all required values. See [Section 4](#) for details.

3.1.3 Deploy

```
bash deploy.sh
```

This script: 1. Loads all Docker images from `images.tar.gz` into the local Docker store 2. Creates any configured host data directories 3. Starts all containers in the background 4. Runs Django database migrations automatically

3.1.4 Create the First Admin User

```
bash deploy.sh createsuperuser
```

3.2 Installing from Source

3.2.1 Clone the Repository

```
git clone <repository-url> isotopeiq-satellite
cd isotopeiq-satellite
```

3.2.2 Create the Environment File

```
cp .env.example .env
```

Edit `.env` and fill in all required values before proceeding. See [Section 4](#) for details.

3.2.3 Build and Start

```
./deploy.sh up
```

This command: 1. Builds all Docker images from source 2. Starts all containers in the background 3. Waits for PostgreSQL to become ready 4. Runs all Django database migrations automatically

3.2.4 Create the First Admin User

```
./deploy.sh createsuperuser
```

Follow the prompts to set a username, email address, and password. This account has full administrative access to the UI and API.

3.3 Accessing the Application

All traffic is routed through the nginx reverse proxy. Direct service ports are not exposed externally.

Interface	URL
Web UI	https://<host>/
REST API	https://<host>/api/v1/
API Auth	https://<host>/api/v1/auth/token/
Django Admin	https://<host>/admin/

Note: A self-signed TLS certificate is generated automatically on first start. To use a trusted certificate, place `server.crt` and `server.key` in the directory configured by `DATA_TLS` (default `./data/tls/`) before starting, and the nginx container will use them in preference to generating a new one.

4. Environment Configuration

All runtime configuration is supplied via the `.env` file. **Never commit this file to version control.**

4.1 Required Settings

Variable	Description	Example
<code>SECRET_KEY</code>	Django secret key — must be long and random	See below
<code>FIELD_ENCRYPTION_KEY</code>	Fernet key for encrypting stored credentials	See below
<code>DB_PASSWORD</code>	PostgreSQL password	<code>s3cur3p@ssw0rd</code>

Variable	Description	Example
ALLOWED_HOSTS	Comma-separated list of allowed hostnames/IPs	satellite.corp.example.com,10.0.1.50

Generate **SECRET_KEY** :

```
python3 -c "import secrets; print(secrets.token_urlsafe(64))"
```

Generate **FIELD_ENCRYPTION_KEY** :

```
python3 -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())"
```

Important: If **FIELD_ENCRYPTION_KEY** changes after credentials have been stored, all encrypted fields (passwords, SSH keys, tokens) become unreadable. Back up the key and treat it with the same care as a certificate private key.

4.2 Database Settings

```
DB_NAME=isotopeiq
DB_USER=isotopeiq
DB_PASSWORD=<strong-password>
DB_HOST=db          # container name; do not change unless using an external DB
DB_PORT=5432
```

4.3 Redis / Celery Broker

```
REDIS_URL=redis://redis:6379/0
```

Change the host from **redis** only if pointing to an external Redis instance.

4.4 Satellite URL

```
SATELLITE_URL=http://localhost:8000
```

Set this to the URL that managed devices can reach when using Push mode. For external devices this must be a publicly routable address or hostname.

4.5 Syslog Notifications

```
SYSLOG_HOST=your-siem.corp.example.com
SYSLOG_PORT=514
```

```
SYSLOG_FACILITY=local0
```

These env-var defaults are applied on first start. Syslog, email, and FTP/SFTP notification settings can also be configured at runtime via **Administration** → **System Settings** in the UI — UI changes take effect immediately without a restart. See [Section 7](#) for details.

4.6 LDAP (Optional)

See [Section 6.2](#).

4.7 SAML 2.0 (Optional)

See [Section 6.3](#).

5. Starting and Stopping Services

All lifecycle operations are handled through `deploy.sh`.

```
Usage: ./deploy.sh [COMMAND] [SERVICE]
```

Command	Description
<code>./deploy.sh up</code>	Build images, start all services, run migrations
<code>./deploy.sh down</code>	Stop and remove containers (data volumes preserved)
<code>./deploy.sh restart</code>	Full stop + start
<code>./deploy.sh restart <service></code>	Restart a single service
<code>./deploy.sh build</code>	Rebuild all images without starting
<code>./deploy.sh logs</code>	Follow logs for all services
<code>./deploy.sh logs <service></code>	Follow logs for one service
<code>./deploy.sh status</code>	Show running container status
<code>./deploy.sh migrate</code>	Run Django migrations manually
<code>./deploy.sh createsuperuser</code>	Create a Django admin user
<code>./deploy.sh shell</code>	Open a bash shell in the backend container
<code>./deploy.sh reset-db</code>	DESTRUCTIVE: destroy and recreate the database volume

Valid service names: `db`, `redis`, `backend`, `frontend`, `worker`, `beat`

Examples:

```
# Restart only the Celery worker after a code change
./deploy.sh restart worker

# Tail backend logs
./deploy.sh logs backend

# Check all container health
./deploy.sh status
```

6. Authentication Configuration

6.1 Local Authentication

Local authentication is enabled by default. Create accounts via the `createsuperuser` command or through the Django admin interface at `http://<host>:8000/admin/`.

JWT tokens are issued at `/api/v1/auth/token/` and refreshed at `/api/v1/auth/token/refresh/`. The web UI handles token management automatically.

6.2 LDAP

LDAP authentication can be configured either through `.env` (applied at startup) or at runtime via **Administration** → **System Settings** → **LDAP Authentication** in the UI. UI changes take effect immediately without a restart. The `.env` values serve as bootstrap defaults.

```
# LDAP server
LDAP_SERVER_URI=ldap://dc01.corp.example.com:389
LDAP_BIND_DN=cn=satellite-readonly,ou=service-accounts,dc=corp,dc=example,dc=com
LDAP_BIND_PASSWORD=<service-account-password>
LDAP_START_TLS=False           # Set True to use STARTTLS (recommended for production)

# User lookup
LDAP_USER_SEARCH_BASE=ou=users,dc=corp,dc=example,dc=com
LDAP_USER_SEARCH_FILTER=(sAMAccountName=%(user)s)    # Active Directory example
# LDAP_USER_SEARCH_FILTER=(uid=%(user)s)              # OpenLDAP example

# Group membership (for role assignment)
LDAP_GROUP_SEARCH_BASE=ou=groups,dc=corp,dc=example,dc=com
LDAP_SUPERUSER_GROUP=cn=satellite-admins,ou=groups,dc=corp,dc=example,dc=com
LDAP_STAFF_GROUP=cn=satellite-users,ou=groups,dc=corp,dc=example,dc=com

# Attribute mapping
LDAP_ATTR_FIRST_NAME=givenName
```

```
LDAP_ATTR_LAST_NAME=sn
LDAP_ATTR_EMAIL=mail
```

Role mapping:

LDAP Group	Django Role	Access Level
LDAP_SUPERUSER_GROUP	Superuser	Full admin access
LDAP_STAFF_GROUP	Staff	UI access, no admin panel

After updating `.env`, restart the backend:

```
./deploy.sh restart backend
```

6.3 SAML 2.0

SAML 2.0 configuration also lives entirely in `.env`.

```
# Service Provider (Satellite) identity
SAML_SP_ENTITY_ID=https://satellite.corp.example.com/saml2/metadata/
SAML_ACS_URL=https://satellite.corp.example.com/saml2/acs/
SAML_SLS_URL=https://satellite.corp.example.com/saml2/ls/

# Identity Provider metadata – use URL or local file (not both)
SAML_IDP_METADATA_URL=https://idp.corp.example.com/metadata
# SAML_IDP_METADATA_FILE=/path/to/idp-metadata.xml

# SP key and certificate (optional; required for signed requests)
SAML_SP_KEY_FILE=/run/secrets/saml_sp.key
SAML_SP_CERT_FILE=/run/secrets/saml_sp.crt
```

SP metadata for registering with the IdP is available at:

```
https://satellite.corp.example.com/saml2/metadata/
```

Register this URL (or download and upload the XML) in your IdP's application configuration.

After updating `.env`, restart the backend:

```
./deploy.sh restart backend
```

7. Notification Configuration

IsotopeIQ Satellite supports three notification/export destinations: **syslog**, **email (SMTP)**, and **FTP/SFTP**. All three can be configured via **Administration** → **System Settings** in the UI — changes take effect immediately without a restart.

For `.env` bootstrap defaults (applied on first start only):

```
# Syslog
SYSLOG_HOST=siem.corp.example.com
SYSLOG_PORT=514
SYSLOG_FACILITY=local0
```

Email and FTP/SFTP do not have `.env` defaults — configure them via the UI.

Syslog

Drift events, new baselines, and collection results are sent as syslog messages when the corresponding **Post-Collection Action** on a policy is triggered. The message includes the device name, policy, and a summary of changed fields.

To verify syslog notifications are reaching your SIEM, trigger a manual collection on a device, modify a monitored value (e.g., add a test user), and re-run. The resulting drift event will generate a syslog message.

Email

Email notifications use a standard SMTP relay. Configure SMTP host, port, STARTTLS, credentials, from address, and recipients in the UI. A post-collection action on the policy must reference the `email` destination for messages to be sent.

FTP / SFTP Export

Baseline snapshots (canonical JSON) can be exported to an FTP or SFTP server automatically after each successful collection. Configure the protocol, host, port, credentials, and remote path in the UI. A post-collection action on the policy must reference the `ftp` destination.

Post-Collection Actions

Notification and export destinations are activated per-policy via **Post-Collection Actions** (set when creating or editing a policy). Each action is a trigger/destination pair:

Trigger	When it fires
<code>new_baseline</code>	First successful collection for a device
<code>drift_detected</code>	Collection differs from the stored baseline
<code>always</code>	Every successful collection

If no post-collection actions are configured on a policy, no notifications are sent regardless of the destination settings.

After changing `.env` syslog settings (bootstrap only), restart the backend:

```
./deploy.sh restart backend
```

8. Service Management Reference

8.1 Service Descriptions

Service	Container	Notes
db	iq-db	PostgreSQL 16; data stored in Docker volume <code>postgres_data</code>
redis	iq-redis	Redis 7; no persistent storage by default
backend	iq-backend	Django app + Gunicorn on port 8000; debugpy on 5678
worker	iq-worker	Celery worker; runs collection jobs; concurrency=4
beat	iq-beat	Celery beat scheduler; triggers policy schedules
frontend	iq-frontend	Vite dev server on port 5173

8.2 Viewing Logs

```
# All services
./deploy.sh logs

# Single service
./deploy.sh logs backend
./deploy.sh logs worker
./deploy.sh logs beat
```

8.3 Running Django Management Commands

```
# Open a shell inside the backend container
./deploy.sh shell

# Or run a command directly
docker compose exec backend python manage.py <command>
```

Common management commands:

```
# Apply new migrations after an upgrade
docker compose exec backend python manage.py migrate

# Check for issues
docker compose exec backend python manage.py check

# Clear expired tokens
docker compose exec backend python manage.py flushexpiredtokens
```

8.4 Celery Worker Concurrency

The Celery worker defaults to 4 concurrent job slots. To change this, edit `docker-compose.yml` and update the `--concurrency` argument on the `celery_worker` service, then rebuild:

```
./deploy.sh restart worker
```

9. Data Retention

Retention settings control how long raw collection output, parsed JSON, job history, and logs are kept. They are stored in the database and configurable via **Administration** → **System Settings** → **Data Retention** in the UI, or via the REST API at `/api/v1/retention/`. Changes take effect on the next pruning run.

Setting	Default	Description
Raw data retention	90 days	Raw text output from collection scripts
Parsed data retention	365 days	Normalized canonical JSON
Job history retention	180 days	Job execution records
Log retention	90 days	System and audit logs

Set any value to **0** to retain data indefinitely.

The retention pruning job runs automatically each day at **03:00 UTC**. No manual action is required after setting the values.

10. Upgrading

1. Pull the latest code:

```
bash git pull
```

1. Rebuild images and restart (migrations run automatically):

```
bash ./deploy.sh up
```

If only the backend changed and you want to avoid frontend rebuild:

```
bash ./deploy.sh restart backend ./deploy.sh restart worker ./deploy.sh restart beat
```

1. Verify services are healthy:

```
bash ./deploy.sh status ./deploy.sh logs backend
```

11. Backup and Restore

11.1 Database Backup

```
# Dump the database to a file
docker compose exec db pg_dump -U isotopeiq isotopeiq > backup-$(date +%Y%m%d).sql
```

11.2 Database Restore

```
# Stop the application services (leave db running)
docker compose stop backend celery_worker celery_beat frontend

# Restore
cat backup-20260329.sql | docker compose exec -T db psql -U isotopeiq isotopeiq

# Restart
./deploy.sh up
```

11.3 Backing Up the Encryption Key

Back up `.env` (especially `FIELD_ENCRYPTION_KEY`) to a secure secrets store (e.g., HashiCorp Vault, AWS Secrets Manager). Without the encryption key, all stored credentials in the database are unreadable.

12. Troubleshooting

Services fail to start

```
./deploy.sh logs
```

Check for missing `.env` values. The most common causes are: - `FIELD_ENCRYPTION_KEY` is empty - `SECRET_KEY` is set to the default placeholder - `DB_PASSWORD` mismatch between the `db` and `backend`

PostgreSQL not ready

If the backend starts before PostgreSQL is ready, the `deploy.sh up` command retries automatically up to 20 times (1 second apart). If it still fails:

```
./deploy.sh logs db
```

Look for disk space or permission errors in the volume.

Migrations fail

```
./deploy.sh logs backend
./deploy.sh migrate
```

If there are conflicting migrations from a failed upgrade, open a shell and inspect:

```
./deploy.sh shell
python manage.py showmigrations
```

Collection jobs fail

1. Check worker logs for the error message:

```
bash ./deploy.sh logs worker
```

1. Common causes:
2. Credential incorrect — verify the username/password or key is correct for the target device
3. Network path blocked between Satellite and device — confirm the relevant port (22, 5985/5986, 161) is reachable
4. Collection script syntax error — test the script manually against a device via the API or a direct SSH session
5. Parser raised an exception — the full traceback appears in the worker logs

Agent Pull devices not responding

- Confirm the agent is running on the device (`systemctl status isotopeiq-agent` / Task Scheduler / launchctl)
- Verify Satellite can reach the device on TCP port 9322 (or the configured agent port)
- Confirm the Agent Secret in **Administration** → **System Settings** matches the secret embedded in the agent config on the device — if the secret was rotated, agents must be reinstalled
- Check backend logs for errors on `GET /collect` requests to the device

LDAP login fails

```
./deploy.sh logs backend
```

Look for `django_auth_ldap` log entries. Common issues: - Incorrect `LDAP_USER_SEARCH_FILTER` for your directory schema - Service account (`LDAP_BIND_DN`) lacks search permission - `LDAP_START_TLS=True` but server certificate is not trusted — mount the CA certificate into the container

SAML login fails

Check that: - `SAML_SP_ENTITY_ID` , `SAML_ACS_URL` , and `SAML_SLS_URL` match exactly what is registered in the IdP - The IdP metadata URL is reachable from the backend container - System clocks on both Satellite and IdP are synchronized (SAML assertions are time-sensitive)

13. Security Hardening

Rotate the Encryption Key

If `FIELD_ENCRYPTION_KEY` is ever exposed:

1. Decrypt all credentials before rotation (export or document them)
2. Generate a new key: `python3 -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())"`
3. Update `.env`
4. Re-enter all credentials in the UI (they will be re-encrypted with the new key)

Network Exposure

- The frontend (5173) and backend (8000) ports should not be exposed directly to untrusted networks. Place a TLS-terminating reverse proxy (nginx, Caddy, HAProxy) in front of both.
- The database (5432) and Redis (6379) ports are not exposed outside Docker by default — do not change this.
- For the push API endpoint (`/api/push/`), ensure the device-to-Satellite path uses HTTPS.

Credential Scope

- Create a dedicated read-only LDAP service account for `LDAP_BIND_DN` . Do not use a domain admin account.
- Use SSH key authentication for managed devices wherever possible rather than passwords.
- Assign devices to credentials with the minimum required privilege (e.g., a user with only `sudo /usr/bin/cat` rather than full sudo).

Audit Log Review

The audit log is queryable via the REST API at `/api/v1/audit-logs/` . Review it regularly for: - Unexpected logins (source IPs, off-hours access) - Credential or device deletions - Script modifications (which could alter what

data is collected)

Updates

Keep the platform updated and monitor for security advisories in the dependencies (Django, Celery, paramiko, pywinrm, djangosaml2).

14. Architecture Reference

Service Interaction

```

Browser
|
▼
frontend (Vite / Vue 3) :5173
|   REST + WebSocket
▼
backend (Django / DRF) :8000
├─ PostgreSQL (db)      :5432  – persistent data store
├─ Redis (redis)        :6379  – Celery broker + result backend
├─ celery_worker         – executes collection jobs
└─ celery_beat           – triggers scheduled policies
  
```

Collection Data Flow

```

Policy schedule fires (or manual Run Now)
→ celery_beat enqueues a job per device
→ celery_worker picks up each job
→ executes Bundle steps in order:
    Client step → runs on device (SSH / Telnet / WinRM / HTTPS)
    Server step → runs on Satellite, receives previous step output via stdin
    Agent Pull  → Satellite calls GET device:9322/collect; no script step needed
→ each step can pipe output to the next
→ step with Enable Baseline: canonical JSON saved as device Baseline
→ step with Enable Drift: output diffed against stored Baseline
    → DriftEvent created and notification sent if differences found
    → Post-Collection Actions evaluated (syslog / email / FTP)
  
```

Key Directories

Path	Contents
backend/apps/	Django application modules (devices, jobs, drift, etc.)
backend/core/	Collection engine, parser engine, canonical schema
backend/config/settings/	Django settings (base, development, production)
frontend/src/	Vue 3 application source
agents/	Pre-built collector binaries and source (Linux, Windows, macOS)
examples/	Example collector and parser scripts for 8+ device types

Agent Binaries

Pre-compiled agents are available for push-mode or deployment scenarios:

Binary	Platform
agents/linux_collector_amd64	Linux x86-64 (glibc 2.17+, RHEL 7+)
agents/linux_collector_i686	Linux x86 32-bit
agents/macos_collector	macOS (recent versions)
agents/windows_collector.exe	Windows 7 32-bit and later

When the managed device has network access to the Satellite, agents can be downloaded directly:

```
GET /api/v1/agents/download/<filename>
```

Python source equivalents are available in `agents/*.py` for environments where running a compiled binary is not permitted.

See [Section 15](#) for installing agents on air-gapped or internet-restricted devices.

15. Agent Installation (Offline)

Use this section when managed devices **cannot reach the Satellite or the internet** to download the agent binary automatically. All required files ship with the Satellite and are pre-placed under `agents/` and `agents/installers/`.

15.1 Files to Transfer to the Managed Device

Copy the following files from the Satellite host to the target device using whatever out-of-band mechanism is available (USB, SCP from a jump host, shared network drive, etc.).

Linux:

File	Source path on Satellite
Agent binary (x86-64)	agents/linux_collector_amd64
Agent binary (32-bit)	agents/linux_collector_i686
Installer script	agents/installers/linux_install.sh
Config file	Download from the device record in the UI (see 15.2)

macOS:

File	Source path on Satellite
Agent binary	agents/macos_collector
Installer script	agents/installers/macos_install.sh
Config file	Download from the device record in the UI (see 15.2)

Windows:

File	Source path on Satellite
Agent binary	agents/windows_collector.exe
Installer script	agents/installers/windows_install.bat
Config file	Download from the device record in the UI (see 15.2)

15.2 Generating the Config File

The config file (`isotopeiq-agent.conf`) ties the agent to a specific device record and contains the device token and Satellite address. Generate it from the UI before transferring files:

1. Go to **Configuration** → **Devices** and open the device record.
2. Set **Connection Type** to **Agent Pull**.
3. Save the device.
4. Retrieve the **Agent Secret** from **Administration** → **System Settings** → **Agent Security**.
5. Use the secret and the Satellite address to build the `isotopeiq-agent.conf` file (see format below).

The config file looks like this:

```
secret=<agent-secret>
server=http://<satellite-ip>:8000
port=9322
```

Note: The `server` value must be reachable from the managed device. On an isolated network, use the Satellite's LAN IP rather than `localhost`. If the Agent Secret is rotated in System Settings, all agent config files must be updated and the agent service restarted.

15.3 Installing on Linux (Offline)

Place all three files in the same directory on the target device, then run:

```
sudo bash linux_install.sh isotopeiq-agent.conf linux_collector_amd64
```

For 32-bit systems:

```
sudo bash linux_install.sh isotopeiq-agent.conf linux_collector_i686
```

The installer: - Copies the binary to `/usr/local/bin/isotopeiq-agent` - Writes the config to `/etc/isotopeiq-agent.conf` (root-read-only) - Creates and enables a systemd unit `isotopeiq-agent.service` that starts at boot

Verify the agent is running:

```
systemctl status isotopeiq-agent
journalctl -u isotopeiq-agent -f
```

15.4 Installing on macOS (Offline)

Place all three files in the same directory, then run:

```
sudo bash macos_install.sh isotopeiq-agent.conf macos_collector
```

The installer: - Copies the binary to `/usr/local/bin/isotopeiq-agent` - Writes the config to `/etc/isotopeiq-agent.conf` (root-read-only) - Installs and loads a launchd daemon `com.isotopeiq.agent` that persists across reboots

Verify the agent is running:

```
sudo launchctl list com.isotopeiq.agent
tail -f /var/log/isotopeiq-agent.log
```

15.5 Installing on Windows (Offline)

Place all three files in the same directory, then run from an **elevated (Administrator) command prompt**:

```
windows_install.bat isotopeiq-agent.conf windows_collector.exe
```

The installer: - Copies the binary to `C:\Program Files\IsotopeIQ\isotopeiq-agent.exe` - Writes the config to `C:\ProgramData\IsotopeIQ\agent.conf` (SYSTEM/Administrators read-only) - Registers a scheduled task `IsotopeIQAgent` that runs as SYSTEM at every system startup - Opens an inbound Windows Firewall rule for

the agent port (default 9322) on domain and private profiles - Starts the agent immediately without requiring a reboot

Verify the agent is running:

```
schtasks /query /tn IsotopeIQAgent /fo LIST /v
type C:\ProgramData\IsotopeIQ\logs\isotopeiq-agent.log
```

15.6 Default Agent Port

All agents listen on TCP port **9322** by default. To use a different port, edit `isotopeiq-agent.conf` before running the installer:

```
port=9500
```

Ensure the chosen port is open inbound on the managed device's firewall and that the Satellite can reach the device on that port.

15.7 Uninstalling

Linux:

```
sudo systemctl disable --now isotopeiq-agent
sudo rm /usr/local/bin/isotopeiq-agent /etc/isotopeiq-agent.conf /etc/systemd/system/isoto
sudo systemctl daemon-reload
```

macOS:

```
sudo launchctl unload /Library/LaunchDaemons/com.isotopeiq.agent.plist
sudo rm /Library/LaunchDaemons/com.isotopeiq.agent.plist
sudo rm /usr/local/bin/isotopeiq-agent /etc/isotopeiq-agent.conf
```

Windows (elevated command prompt):

```
schtasks /end /tn IsotopeIQAgent
schtasks /delete /tn IsotopeIQAgent /f
netsh advfirewall firewall delete rule name="IsotopeIQ Agent"
rmdir /s /q "C:\Program Files\IsotopeIQ"
rmdir /s /q "C:\ProgramData\IsotopeIQ"
```

15.8 Using the Python Source Instead of a Binary

If policy prevents running compiled binaries, the Python source agents in `agents/*.py` can be used directly. The source agents have the same CLI interface as the compiled binaries:

```
# Linux / macOS
python3 agents/linux_collector.py --serve --port 9322

# Windows (PowerShell)
python agents\windows_collector.py --serve --port 9322
```

Python 3.8 or later is required. Install dependencies first:

```
pip3 install requests cryptography
```

The installer scripts do not support the Python source directly — run the agent manually or wrap it in your own service definition.